

Day 3: SOLID 리팩토링 & Spring Boot REST API 구축

목표: 수동 객체 주입을 Spring DI(`@Autowired`, `@Service`, `@Component`)로 자동화하고 REST API 컨트롤러를 구축합니다.

1. Spring Boot 이식 (`app/build.gradle`)

`app/build.gradle` 파일 전체를 아래 코드로 수정합니다.

```
plugins {  
    id 'java' // 자바 프로젝트 기본 빌드 기능 제공  
    id 'org.springframework.boot' version '3.2.0' // Spring Boot 의존성 및 태  
스크 관리 플러그인 설정  
    id 'io.spring.dependency-management' version '1.1.4' // 의존성 버전을 자동으  
로 관리하는 플러그인  
}  
  
group = 'com.example' // 프로젝트의 그룹 ID 설정  
version = '0.0.1-SNAPSHOT' // 애플리케이션 버전 설정  
java { sourceCompatibility = JavaVersion.VERSION_17 } // Java 17 버전 호환성  
설정  
  
repositories {  
    mavenCentral() // 중앙 마이븐 리포지토리에서 라이브러리 패키지 다운로드  
}  
  
dependencies {  
    // Spring MVC RESTful 웹 서비스를 구축하기 위한 필수 스타터 팩  
    implementation 'org.springframework.boot:spring-boot-starter-web'  
  
    // 비밀번호 암호화를 위한 BCrypt 라이브러리  
    implementation 'org.mindrot:jbcrypt:0.4'  
  
    // JWT 토큰 생성을 위한 인터페이스 라이브러리  
    implementation 'io.jsonwebtoken:jjwt-api:0.11.5'  
  
    // JWT 서명 및 검증 로직 실행 라이브러리 (런타임 시에만 필요)  
    runtimeOnly 'io.jsonwebtoken:jjwt-impl:0.11.5'  
  
    // JWT 생성을 위한 JSON 파싱 라이브러리 (런타임 시에만 필요)  
    runtimeOnly 'io.jsonwebtoken:jjwt-jackson:0.11.5'  
}  
  
// Runnable JAR 생성 시 시작점(Main Class) 위치 명시  
bootJar {  
    mainClass = 'com.example.auth.Application'  
}  
  
// bootRun 실행 시 시작점(Main Class) 위치 명시 (ClassNotFoundException 에러 방지)
```

```
springBoot {  
    mainClass = 'com.example.auth.Application'  
}
```

2. Spring 컴포넌트 선언

1) `TokenProvider.java` 수정

app/src/main/java/com/example/auth/TokenProvider.java

```
package com.example.auth;  
  
import io.jsonwebtoken.Jwts;  
import io.jsonwebtoken.security.Keys;  
import org.springframework.stereotype.Component;  
import java.util.Date;  
import java.security.Key;  
  
/**  
 * JWT(JSON Web Token) 발급 및 관리 전담 클래스  
 * @Component: 스프링 컨테이너에 의해 관리되는 Bean(객체)으로 등록  
 */  
@Component  
public class TokenProvider {  
    // JWT 암호화에 사용할 비밀 키 (최소 32자/256비트 이상 권장)  
    private final String SECRET = "day2-secret-key-must-be-at-least-32-chars!!";  
  
    // HMAC-SHA 규격에 맞춰 Key 객체 생성  
    private final Key key = Keys.hmacShaKeyFor(SECRET.getBytes());  
  
    /**  
     * 사용자 ID를 기반으로 30분 유효기간의 JWT 토큰을 생성합니다.  
     */  
    public String generate(String userId) {  
        return Jwts.builder()  
            .setSubject(userId) // 토큰 주체(Subject)로 사용자 ID 설정  
            .setExpiration(new Date(System.currentTimeMillis() +  
1800000)) // 유효 기간: 현재 시간 기준 30분(1,800,000ms)  
            .signWith(key) // 암호화 키를 활용한 디지털 서명  
            .compact(); // 완성된 JWT 토큰 문자열 반환  
    }  
}
```

2) `AuthService.java` 수정

app/src/main/java/com/example/auth/AuthService.java

```
package com.example.auth;
```

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.HashMap;
import java.util.Map;

/**
 * 회원 가입 및 로그인 비즈니스 로직 처리 클래스
 * @Service: 스프링 내부의 비즈니스 레이어 Bean으로 등록
 */
@Service
public class AuthService {
    // 생성자 주입(DI)을 받을 의존성 객체
    private final TokenProvider tokenProvider;

    // 임시 인메모리 저장소 (ID -> User 객체 매핑)
    private final Map<String, User> store = new HashMap<>();

    /**
     * @Autowired: 스프링 컨테이너가 자동으로 TokenProvider 객체(Bean)를 주입 (생성자 주입)
     */
    @Autowired
    public AuthService(TokenProvider tokenProvider) {
        this.tokenProvider = tokenProvider;
    }

    /**
     * 신규 회원가입 처리
     */
    public void register(String id, String pw) {
        // 이미 저장소에 동일한 ID가 존재하는지 체크
        if (store.containsKey(id)) throw new RuntimeException("이미 존재하는 ID");

        // BCrypt 비밀번호 암호화 후 인메모리 Map에 저장
        store.put(id, User.of(id, pw));
    }

    /**
     * 로그인 검증 및 토큰 발급
     */
    public String login(String id, String pw) {
        User user = store.get(id);

        // 사용자 아이디가 없거나, 비밀번호가 검증을 통과하지 못하면 예외 발생
        if (user == null || !user.matchPw(pw)) {
            throw new RuntimeException("인증 실패");
        }

        // 정상 검증 시 TokenProvider를 이용해 JWT 토큰 생성 후 반환
        return tokenProvider.generate(id);
    }
}
```

3. REST API 컨트롤러 및 Spring Boot 메인 클래스

1) **AuthController.java** 작성

app/src/main/java/com/example/auth/AuthController.java

```
package com.example.auth;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

/**
 * HTTP 엔드포인트를 노출하고 클라이언트 요청을 수신하는 컨트롤러
 * @RestController: @Controller + @ResponseBody의 조합으로, 메서드의 반환값을 JSON
형태의 HTTP 응답 본문으로 직접 출력
 * @RequestMapping: 해당 클래스의 모든 메서드 기본 URL 경로를 지정 ("/api/auth")
 */
@RestController
@RequestMapping("/api/auth")
public class AuthController {

    /**
     * @Autowired: 필드 주입을 통한 AuthService 의존성 자동 연결
     */
    @Autowired
    private AuthService authService;

    // HTTP 요청 바디(JSON)를 맵핑받기 위한 DTO (Java Record 구문 활용)
    public record AuthRequest(String id, String password) {}

    // 클라이언트에게 토큰을 JSON 형태로 응답하기 위한 DTO
    public record TokenResponse(String token) {}

    /**
     * 회원가입 API 엔드포인트
     * @PostMapping: POST 요청 처리
     ("http://localhost:8080/api/auth/register")
     * @RequestBody: 요청 본문(JSON)을 Java 객체(AuthRequest)로 변환
     */
    @PostMapping("/register")
    public ResponseEntity<String> register(@RequestBody AuthRequest req) {
        // 회원가입 비즈니스 로직 실행
        authService.register(req.id(), req.password());

        // HTTP Status 201(Created) 상태 코드와 응답 메시지 전달
        return ResponseEntity.status(201).body("가입 완료");
    }

    /**
     * 로그인 API 엔드포인트

```

```

    * @PostMapping: POST 요청 처리 ("http://localhost:8080/api/auth/login")
    */
    @PostMapping("/login")
    public ResponseEntity<TokenResponse> login(@RequestBody AuthRequest
req) {
        // 인증 수행 후 JWT 토큰 발급받음
        String token = authService.login(req.id(), req.password());

        // HTTP Status 200(OK)과 함께 발급받은 JWT 토큰을 JSON 객체로 전달
        return ResponseEntity.ok(new TokenResponse(token));
    }
}

```

2) Application.java 작성 (기존 App.java 대체)

app/src/main/java/com/example/auth/Application.java

```

package com.example.auth;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

/**
 * @SpringBootApplication: Spring Boot 구동의 중심이 되는 어노테이션
 * 1) @SpringBootConfiguration: 스프링 환경 설정 등록
 * 2) @EnableAutoConfiguration: 하위 패키지의 의존성 라이브러리 기반 자동 설정(웹서버
등)을 활성화
 * 3) @ComponentScan: 현재 패키지 및 하위 패키지 내의 @Component, @Service,
@RestController 등을 탐색하여 Bean으로 등록
 */
@SpringBootApplication
public class Application {
    public static void main(String[] args) {
        // 내장 톰캣(Tomcat) 서버를 구동하고 Spring 애플리케이션 컨테이너를 실행
        SpringApplication.run(Application.class, args);
    }
}

```

4. Gradle 업데이트 및 실행 오류 방지 설정

빌드 에러(LenientConfiguration, Configuration Cache, 메인 클래스 로딩 실패)를 방지하기 위해 Gradle Wrapper를 최신 버전으로 업그레이드하고 실행합니다.

```

# 1. Gradle Wrapper를 8.5 버전으로 업그레이드
./gradlew wrapper --gradle-version 8.5

# 2. 기존 빌드 캐시 삭제
./gradlew clean

```

```
# 3. 캐시 옵션을 비활성화하여 Spring Boot 실행
./gradlew bootRun --no-configuration-cache
```

5. API 테스트

- 회원가입 요청: `POST http://localhost:8080/api/auth/register` (Body: JSON `{"id":"alice", "password":"123"}`)
- 로그인 요청: `POST http://localhost:8080/api/auth/login` → 토큰 발급 확인